

기술의 원리를 이해하고 적용하는 백엔드 개발자 정명진

단순히 사용하는 것을 넘어 왜 사용하는지 고민합니다.

CONTACT

☎ phone : 010-2509 -8557 | ✉ Mail : bnm070289@gmail.com

JEONG MYOUNGJIN

GitHub : <https://github.com/jmj010702>

01. 자기소개

- 프로필 · 기술 스택 (P.03)
-

02. PEOPLECORE 피플코어 (B2B HR 인사 시스템)

- 프로젝트 소개 및 주요 역할 (p.04)
 - 프로젝트 성과 (p.05)
 - 트러블 슈팅 및 회고 (p.06)
-

03. POCHAON 포차온 (실시간 테이블 주문 POS 서비스)

- 프로젝트 소개 및 주요 역할 (p.07)
 - 프로젝트 성과 (p.08)
 - 트러블 슈팅 및 회고 (p.09)
-



정명진

JEONG MYUNGJIN

📅 2001.07.02 (만 24세)

☎ 010-2509-8557

✉ bnm070289@gmail.com

📍 서울, 경기 거주

📍 육군 병장 만기전역

학력 및 교육 사항

2025.11 - 2026.05 한화 SW CAMP 백엔드 과정 수료

2020.02 - 2026.02 나사렛대학교 IT융합학부 졸업

자격사항 및 수상내역

2026.03 PCCE Lv3

2026.01 정보처리 기사 필기 취득

2020.03 운전면허 1종

기타 활동

2025.09 교내 캡스톤 학술제 - ONEROOM RECIPE

2024.09 교내 캡스톤 학술제 - AUTO DIALOG

2023.09 교내 캡스톤 학술제 - AUTO AIR TICKET

기술 스택

언어 Java · Python

프레임워크 Spring Boot · Spring Batch · JPA · QueryDSL · FastAPI

DB MySQL · Redis · Elasticsearch · Oracle · PostgreSQL · MinIO

메시징 Kafka · WebSocket · Debezium · SSE

인프라 AWS EKS · ECR · S3 · Docker · GitHub Actions · GitHub Packages · EC2

기타 React · Claude · Vue.js, RAG

활용 도구

개발도구 IntelliJ IDEA · VS Code · SourceTree

협업도구 Notion · GitHub · Discord · Figma

핵심 역량

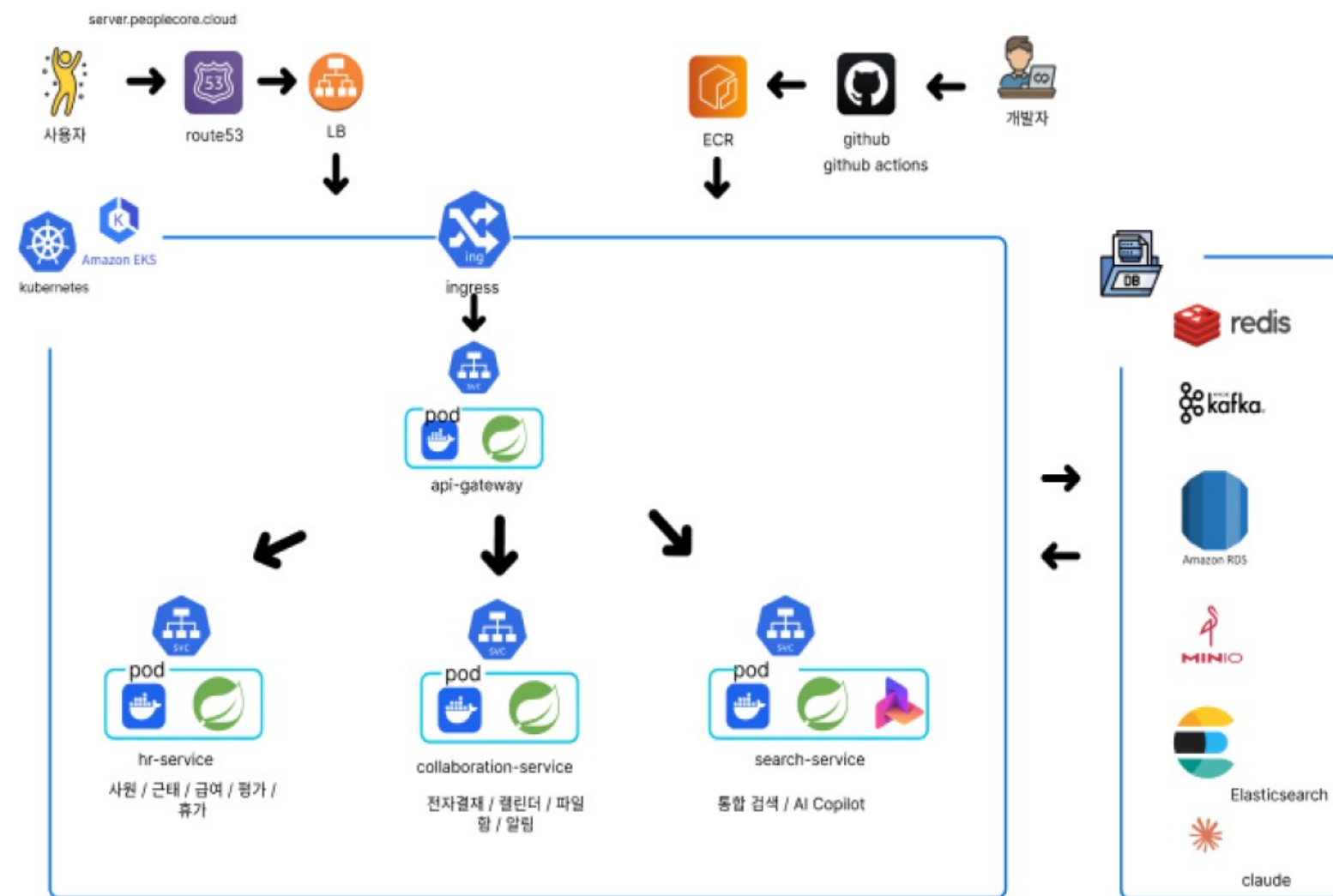
- 새로운 기술을 빠르게 습득하고 실무에 적용 가능

- 주도적인 커뮤니케이션으로 문제를 해결

PEOPLECORE

피플코어

- <https://github.com/beyond-sw-camp/be23-fin-2team-PeopleCore-be>
- <https://github.com/beyond-sw-camp/be23-fin-2team-PeopleCore-fe>

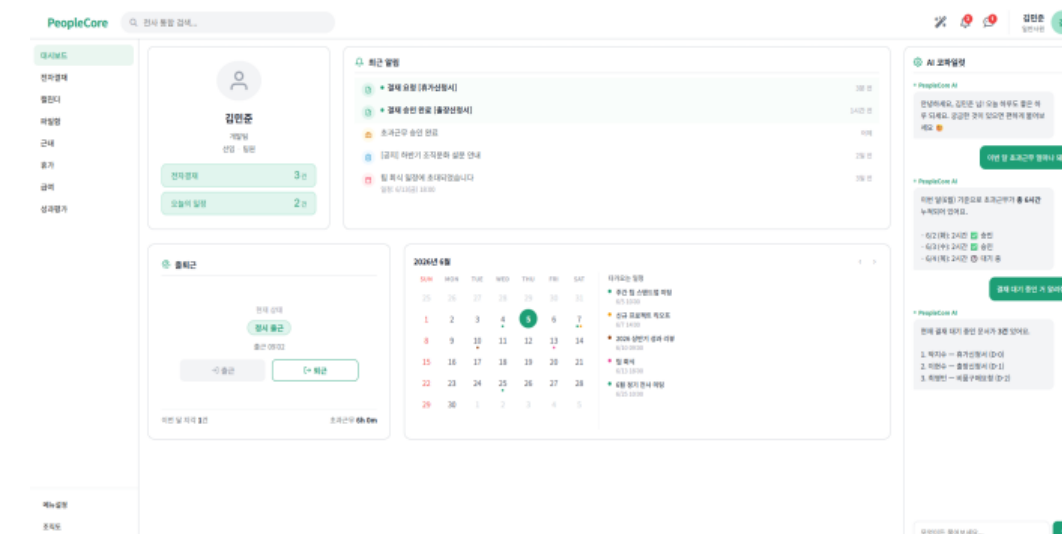


프로젝트 소개

- 기업의 인사•결재•근태 업무를 하나의 플랫폼으로 통합한 B2B HR SaaS 프로그램
- 회사별 결재선, 근무 정책, 급여 항목 등을 커스터마이징할 수 있도록 설계
- 도메인별 마이크로서비스 분리로 독립 배포와 장애 격리가 가능한 구조 구현

주요 역할

- 5개 마이크로서비스 분리 및 Gateway/Eureka 기반 백엔드 아키텍처 설계
- 서비스별 데이터를 실시간으로 수집•색인하는 통합검색 구조 구현
- 키워드•의미 기반 하이브리드 검색과 민감도 기반 LLM 라우팅 구조 설계
- 전자결재•휴가•근태 도메인의 핵심 비즈니스 로직 구현
- 연차 배치, 근태 자동마감, 파티셔닝 등 운영 자동화 구조 적용
- AWS를 사용한 백엔드 및 프론트 배포



프로젝트 성과

성과 01·AI 비용

AI 비용 절감

프롬프트 캐싱 적용으로 고정 시스템 프롬프트 재
사용

입력 토큰 비용 83% 절감 · 응답 품질 유지

성과 02·보안

민감 데이터 외부 전송 차단

민감도 자동 분류로 sLLM·Claude 이중 라우팅을
설계해

민감도 기반 모델 라우팅으로 외부 전송 통제

성과 03·데이터 정합성

CDC 기반 자동 동기화 구축

Debezium CDC로 binlog를 자동 감지해 ES 색
인을 자동화하고

RDB 변경 사항을 ES에 실시간 반영

성과 04·동시성

중복 체크인 방지

DB UNIQUE + saveAndFlush 이중 방어와
파티션 키 명시로

멀티 파드 환경에서 체크인 무결성 보장

성과 05·보안

배치 실행 이력 관리 자동화

Quartz JDBC로 배치 실행 이력을 DB에 자동
저장

실패 원인 추적 및 운영 가시성 확보

성과 06·설계

State 패턴 적용

결재 상태별 State 구현체로 분리해 전이 규칙을
캡슐화 하고

상태 전이 로직 표준화 및 유지보수성 향상

트러블슈팅 & 회고

case 01·동시성

문서번호 채번의 동시성

문제

동시 기안 시 문서번호 순번 중복 발급

→ 중복된 문서번호로 같은 문서를 혼동하는 문제 발생

원인

순번을 조회 후 저장하는 사이로 다른 요청이 끼어듦

해결

낙관적 락으로 순번행을 선점하고, 충돌 시 3회 재시도 후 UNIQUE 제약으로 중복 차단

배운점

락 전략을 고르면서 기술에 정답이 없다는 걸 체감했다. 충돌 빈도와 운영 비용을 함께 따지고, 트레이드 오프를 알고 선택하는 것 자체가 설계라는 걸 배움

case 02·멀티서버

멀티서버에서 SSE 알림의 누락

문제

서버 확장 후 다른 서버에 연결된 사용자에게 SSE알림 누락

→ 사용자가 직접 확인 전까지 상황 인지 누락

원인

SSE 세션이 각 서버 메모리에 종속되어 인스턴스 간 공유 불가

해결

Kafka Consumer로 메시지를 수신해 각 서버에서 자신의 SSE 세션에 직접 fan-out

배운점

실시간 통신은 처음부터 분산 환경을 전제하고 설계해야 하는걸 배움

case 03·Tool 미지원

EXAONE Tool Use 미지원

문제

EXAONE 3.5가 네이티브 Tool Use를 지원하지 않아 도구 호출 불가 → AI 기능 구현 불가능해짐

원인

Claude는 tool_use_API를 지원하지만 EXAONE은 일반 텍스트 응답만 가능

해결

[[CALL]] 마커 기반 프로토콜을 직접 설계해 두 LLM이 동일한 도구 인터페이스로 동작

배운점

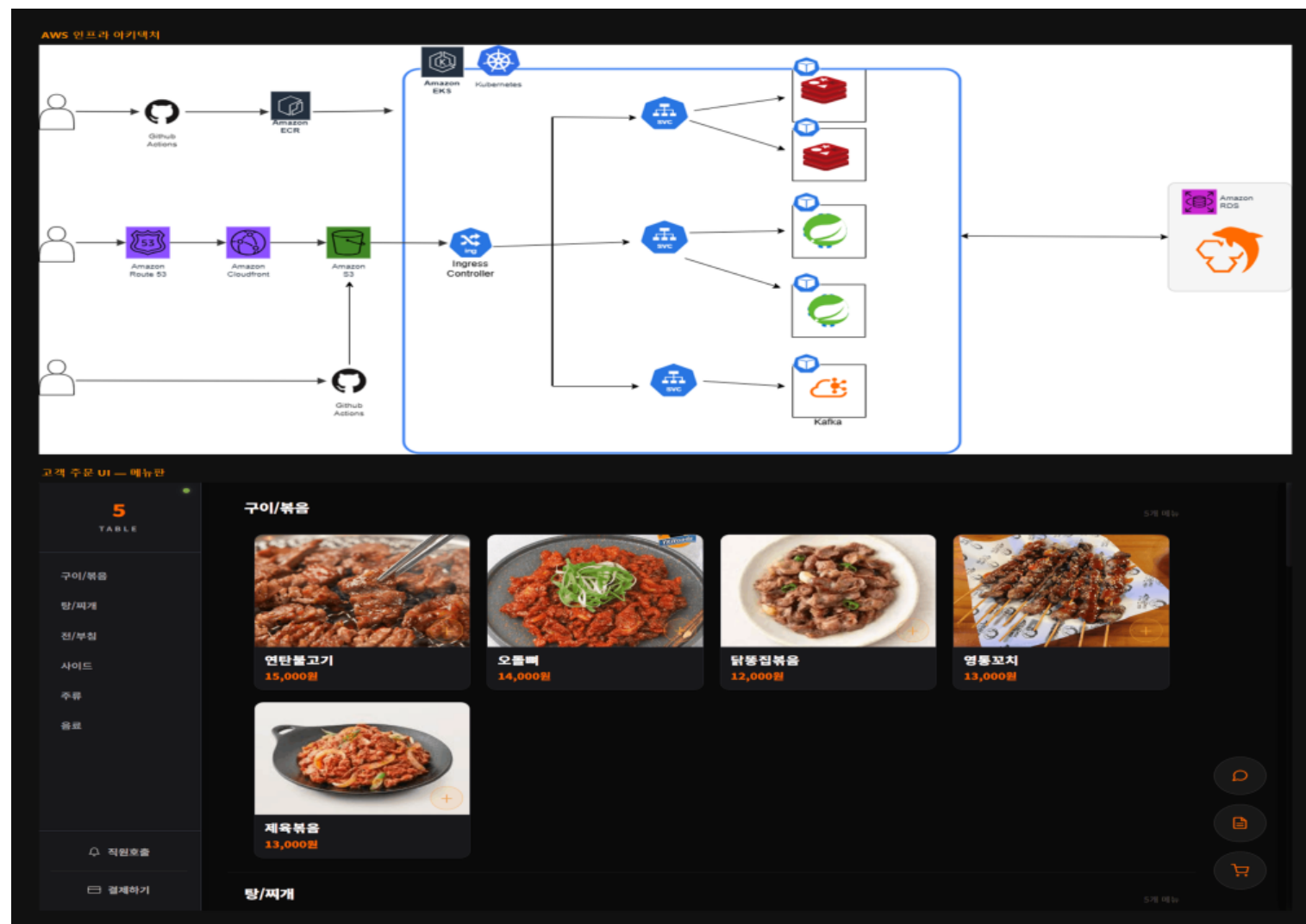
기술이 지원하지 않을 때 도구를 바꾸기 전에 구조로 해결할 수 있는지 먼저 고민해야 된다는 걸 배움

회고 | 문제마다 여러 대안을 비교하면서, 기술 선택의 기준은 성능이 아닌 그 상황의 트레이드오프임을 배웠다

PEOPLECORE

포차온

- <https://github.com/beyond-sw-camp/be23-fin-2team-PeopleCore-be>
- <https://github.com/beyond-sw-camp/be23-fin-2team-PeopleCore-fe>



프로젝트 소개

- 매장과 테이블을 잇는 실시간 오더 시스템, 테이블 간 채팅까지 통합한 B2B POS SaaS 서비스
- 어드민·점주·고객 3역할, 매장별 메뉴·재고 독립 관리 구조

주요 역할

- Kafka 이벤트 기반 실시간 테이블 주문 처리
- 비관적 락 기반 FIFO 식재료 재고 관리
- Toss Payment API 연동을 통한 결제·취소·환불 구현
- SSE + Redis Pub/Sub 기반 실시간 알림
- WebSocket/Stomp을 통한 테이블간 실시간 채팅
- Spring Scheduler 기반 만료 재고 정리·구독 자동 연장 배치
- Redis 멍등성 키 기반 테이블 간 메뉴 선물

프로젝트 성과

성과 01·동시성

재고 정확성 확보

Redis 락 + DB UNIQUE
동시 재고 차감 오류율 75% → 0%

성과 02·실시간성

주문 알림 신뢰성 확보

Kafka DLT + SSE·Pub/Sub 적용
메세지 누락 0%

성과 03·DB 튜닝

94% 응답시간 단축

DB Slow Query로 N+1 탐지 후 해결
1만 건 기준 조회 최대 5초 → 0.2초

성과 04·커넥션 튜닝

주문 처리 에러율 0%

HikariCP pool size 2 → 10 최적화
100건 동시 주문 에러율 79% → 0%, TPS 19% 향상

트러블슈팅 & 회고

case 01·배포

EKS 배포 후 WebSocket 연결 불안정

문제

배포 후 실시간 연결이 30초~1분 간격으로 반복 종료
→ 실시간 주문·채팅 기능 마비

원인

서버가 일정시간 응답 없으면 연결을 강제 종료하는 기본값이 장시간 유지가 필요한 Websocket과 충돌

해결

서버 설정에서 연결 유지 시간을 60초 ~ 1시간으로 연장

배운점

로컬에서 정상 동작해도 인프라 기본값이 서비스 특성과 충돌할 수 있다. 장시간 연결이 필요한 기능은 처음부터 인프라 설계를 해야한다는 걸 배웠다.

case 02·트랜잭션

결제 승인 후 후처리 불일치

문제

결제 완료 후 테이블 초기화·주문 상태 미반영
→ 결제된 테이블에 다음 손님 착석 불가

원인

결제 승인과 후처리가 하나의 흐름으로 묶여 후처리 실패 시 전체 롤백

해결

결제 승인과 후처리를 분리, 결제 성공은 항상 보장하고 후처리는 각각 독립 처리

배운점

외부 결제 API와 내부 후처리는 경계를 분리해 각각 독립 보장해야 한다는 걸 배웠다.

case 03·보안

결제 금액 위변조

문제

프론트에서 결제 금액을 임의로 변조해 실제보다 적은 금액으로 결제 가능
→ 매출 손실 직결

원인

결제 승인 시 프론트가 보낸 금액을 검증 없이 그대로 사용

해결

결제 생성 시 서버 DB에 금액 선저장, 승인 시 DB금액과 불일치하면 즉시 실패 처리

배운점

금액처럼 비즈니스에 직결되는 값은 서버가 단독 보유한 원본과 대조해야 한다는 걸 배웠다.

회고 | 기술 장애가 매출·운영에 직결되는 세 번의 경험을 겪으면서, 코드 안정성은 비즈니스 신뢰와 같다는 걸 배웠다.